

專題名稱: Abnormal Traffic Detection System for Wireless Devices

組員:113064558 謝育葦、113064514 劉姿妤

零. 背景

(1).Botnet(殭屍網路)

Botnet（殭屍網路）是由大量被惡意軟體感染並受到遠端控制的裝置所組成的網路。這些被感染的裝置（稱為「殭屍機」或「bot」）通常包括個人電腦、伺服器，甚至是物聯網設備（如攝影機、路由器等），而它們的擁有者通常並不知道自己的設備已經遭到控制。Botnet 可以用來做 DDoS 攻擊、散播垃圾郵件、挖礦等非法行為。

(2).CNN-LSTM

這次的專題我們採用了 CNN 與 LSTM 的混合架構 [1]。使用這種模型的出發點，是希望 CNN 能擷取每個封包的空間特徵（如欄位間的分佈特性），而 LSTM 則能擷取封包序列的時間特徵（即異常出現的趨勢與順序關聯），使得模型學習到的特徵表示更完整、具備更強的辨識能力。此外，我們在 CNN 與 LSTM 之間加入了 Max Pooling 與 Batch Normalization，前者能進一步強化模型對重要特徵的抽取能力，後者則可避免資料在神經網路中傳遞時產生 Covariance Shift（協變異移轉）的問題，提升模型穩定性與收斂速度。

一. 動機

若我們的主機成為 botnet 的一部分，可能會有電腦變慢、電費暴增、系統提早老化等負面影響。雖然現今防毒軟體已經有一定功能檢測殭屍網路病毒，但傳統防毒軟體主要是透過特徵碼（signature）和靜態掃描來判斷是否中毒，例如檔案內容、系統行為是否與某種已知惡意程式相符。但這樣的方式不容易偵測「正在發起攻擊的主機」，尤其是殭屍網路中的受害者主機，可能沒有明顯中毒徵兆，只是暗中發送異常的封包。因此我們提出的專題針對的是『主機是否參與異常流量攻擊』這種動態行為判斷。它能即時監控並分類封包型態，即使對方使用未知殭屍程式，也能透過異常流量模式偵測出來，這是傳統防毒無法做

到的。

此外，我們之所以選擇使用 CNN-LSTM 的混合架構，主要是基於其能夠同時擷取空間與時間兩種特徵的能力。CNN（Convolutional Neural Network）擅長處理具有區域性結構的數據，能夠有效擷取每一筆封包或每個時間點特徵中的「空間模式」與「局部重要特徵」，例如欄位間的互動、變異幅度、或異常分佈的結構，而 LSTM（Long Short-Term Memory）則是一種專為序列資料設計的遞迴神經網路（RNN），具備長期記憶能力，能夠捕捉資料在時間維度上的變化與關聯，像是異常流量發生前的趨勢、週期性變化、或持續出現的可疑行為。

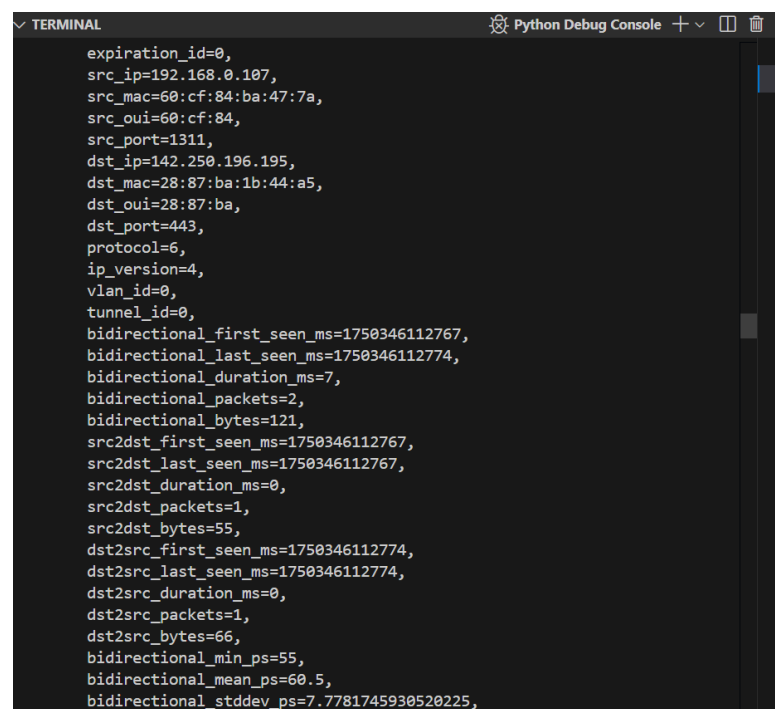
這樣的組合架構讓模型不只侷限於單一時刻的特徵分析，而是能夠在多筆連續資料之間建立動態關聯性，進一步判斷是否存在異常行為。具體來說，CNN 可先將每筆封包或紀錄轉換為高階空間特徵向量，再由 LSTM 依序讀入這些時間點上的特徵，整合成一個代表該段流量序列語意的時間上下文表示，進而提升模型對異常行為的辨識力與準確性。更重要的是，根據我們所參考的文獻 [1] 中的實驗結果，這種 CNN-LSTM 的混合模型在公開的異常流量資料集上展現了極為優異的表現，其準確率、召回率與 F1-score 均明顯優於傳統機器學習方法與單一模型（如單獨的 CNN 或 LSTM）。這也進一步驗證了該架構在處理複雜異常模式時的強大潛力。因此，綜合其理論基礎、資料適配性，以及在文獻中的實際驗證結果，這些都是我們選擇採用 CNN-LSTM 架構作為核心模型的主要動機。我們期望透過這樣的模型設計，讓異常偵測系統在面對實際無線裝置流量的環境中，也能維持高準確率與穩定性。

二. 偵測網路流量

在我們的系統中，一筆 flow（資料流）是指“從同一個來源 IP 和 port 發送到同一個目的 IP 和 port，使用同一種傳輸協定（如 TCP）的連續封包集合。”。我們使用 nfstream 這個套件來幫我們抓取這台主機每一筆進出網卡的 flow，當我們要偵測時，要特別設定網卡的名稱，及每一筆 flow 的 timeout，如下圖這樣

```
streamer = NFStreamer(
    source=interface_name, # 換成你的網卡英文名稱
    statistical_analysis=True,
    decode_tunnels=True,
    idle_timeout=idle_timeout_setting, # 閒置多少秒就輸出 flow
    active_timeout=inactive_timeout_setting # 最長多少秒就輸出 flow
)
```

當我們連接網路並進行活動時，可以像下圖這樣抓取每一筆 flow 的資料並顯示在 terminal 上，



```

✓ TERMINAL Python Debug Console
{
  expiration_id=0,
  src_ip=192.168.0.107,
  src_mac=60:cf:84:ba:47:7a,
  src_oui=60:cf:84,
  src_port=1311,
  dst_ip=142.250.196.195,
  dst_mac=28:87:ba:1b:44:a5,
  dst_oui=28:87:ba,
  dst_port=443,
  protocol=6,
  ip_version=4,
  vlan_id=0,
  tunnel_id=0,
  bidirectional_first_seen_ms=1750346112767,
  bidirectional_last_seen_ms=1750346112774,
  bidirectional_duration_ms=7,
  bidirectional_packets=2,
  bidirectional_bytes=121,
  src2dst_first_seen_ms=1750346112767,
  src2dst_last_seen_ms=1750346112767,
  src2dst_duration_ms=0,
  src2dst_packets=1,
  src2dst_bytes=55,
  dst2src_first_seen_ms=1750346112774,
  dst2src_last_seen_ms=1750346112774,
  dst2src_duration_ms=0,
  dst2src_packets=1,
  dst2src_bytes=66,
  bidirectional_min_ps=55,
  bidirectional_mean_ps=60.5,
  bidirectional_stddev_ps=7.7781745930520225,
}

```

然後我們將偵測流量的程式分成 Record CSV mode 跟 detect mode，

```
#setting
record_csv_mode=False # 是否要把每筆flow寫入csv檔
detect_mode=False # 是否要偵測攻擊流量
```

前者會把抓取的 flow 資料存進 CSV 檔，可作為訓練集。後者是會把我們訓練好的 model 讀取進來，來判別每一筆 flow 是屬於哪一個類別，及搭配其他延伸應用(例如警告彈窗)。

三. Training

(1). 如何製作 training dataset

在製作 training dataset 時，將偵測流量的程式設定成 record CSV mode，我們把 flow 的資訊存進 CSV 檔內，如下圖這樣

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
id	expiration	src_ip	src_mac	src_oui	src_port	dst_ip	dst_mac	dst_oui	dst_port	protocol	ip_version	vlan_id	tunnel_id	bidirection	bidirection	bidirection	bidirection	bidirection	src2dst	dst2src	src2dst	dst2src
0	0	192.168.5.60	cf84:4a:60:cf:84	50425	142.250.110	28:74:4d:f0:28:74	443	17	4	0	0	1.75E+12	1.75E+12	407	20	9623	1.75E+12	1.75E+12	399	10	399	10
1	0	192.168.5.60	cf84:4a:60:cf:84	50425	142.250.110	28:74:4d:f0:28:74	443	17	4	0	0	1.75E+12	1.75E+12	131	14	10088	1.75E+12	1.75E+12	123	8	123	8
2	0	192.168.5.60	cf84:4a:60:cf:84	59647	192.168.5.10	28:74:4d:f0:28:74	53	17	4	0	0	1.75E+12	1.75E+12	7	2	310	1.75E+12	1.75E+12	0	1	0	1
5	0	192.168.5.60	cf84:4a:60:cf:84	1226	103.28.54.10	28:74:4d:f0:28:74	27025	6	4	0	0	1.75E+12	1.75E+12	61	2	168	1.75E+12	1.75E+12	0	1	0	1
6	0	192.168.5.60	cf84:4a:60:cf:84	2591	20.44.10.110	28:74:4d:f0:28:74	443	6	4	0	0	1.75E+12	1.75E+12	857	19	7238	1.75E+12	1.75E+12	688	9	688	9
8	0	192.168.5.60	cf84:4a:60:cf:84	50425	142.250.110	28:74:4d:f0:28:74	443	17	4	0	0	1.75E+12	1.75E+12	282	25	19041	1.75E+12	1.75E+12	274	12	274	12
9	0	192.168.5.60	cf84:4a:60:cf:84	1676	20.50.201.10	28:74:4d:f0:28:74	443	6	4	0	0	1.75E+12	1.75E+12	709	10	890	1.75E+12	1.75E+12	452	4	452	4
11	0	192.168.5.60	cf84:4a:60:cf:84	53588	203.66.18.10	28:74:4d:f0:28:74	443	17	4	0	0	1.75E+12	1.75E+12	44	238	258618	1.75E+12	1.75E+12	44	29	44	29
13	0	192.168.5.60	cf84:4a:60:cf:84	55533	142.250.210	28:74:4d:f0:28:74	443	17	4	0	0	1.75E+12	1.75E+12	24	7	1396	1.75E+12	1.75E+12	24	3	24	3
14	1	192.168.5.60	cf84:4a:60:cf:84	50425	142.250.110	28:74:4d:f0:28:74	443	17	4	0	0	1.75E+12	1.75E+12	284	19	10200	1.75E+12	1.75E+12	276	10	276	10

若是偵測因使用正常活動的網路流量，需標記為 normal

r	dst2src_ac	dst2src_ps	dst2src_rst	dst2src_fir	application	application	application	application	requested_client	finger	server_finger	user_agent	content_ty	Label
)	0	0	0	0	Unknown	Unspecifie	0	0						normal
)	0	0	0	0	Unknown	Unspecifie	0	0						normal
)	0	0	0	0	DNS.Micro	Network	0	6	mobile.events.data.microsoft.com					normal
)	1	0	0	0	TLS	Web	0	6						normal
)	10	4	0	1	TLS.Micro	Cloud	0	6	mobile.ev	d9298114f	15af977ce25de452b96affa2ad	db		normal
)	0	0	0	0	Unknown	Unspecifie	0	0						normal
)	6	2	0	0	TLS	Web	0	6						normal
)	0	0	0	0	Unknown	Unspecifie	0	0						normal
)	0	0	0	0	Unknown	Unspecifie	0	0						normal

異常流量的部分，我們分為兩種方式製造，一種是收集網路上公開的異常流量的 pcap 檔，這些檔案為真實世界和實驗室測試得來的數據。我們將其修改其 IP、port，並利用 nfstream 補捉這筆流量的資訊，另一種方法為我們自己寫一個程式製造異常流量，會一直利用 scapy 對外發送封包，我們的攻擊模式比較偏向 Burst Attack，因此每波攻擊都隨機維持 5~30 秒之間，每波攻擊之間也都會間隔一個隨機的秒數。然後我們是使用目標 IP 和目標 port 來幫助標記異常，要把所有異常流量的資料的目標 IP 都改成攻擊對象的 IP(這邊為虛擬機 IP)，而目標 port 要根據不同攻擊方法(包含 SYN flood、ACK flood、RST flood、FIN flood)來賦予不同的數字。

DNS	MicroNetwork	0	6	mobile.events.data.microsoft.com	normal
TLS	Web	0	6		normal
Unknown	Unspecifie	0	0		normal
Unknown	Unspecifie	0	0		normal
DNS	Network	0	6	p2p-hkg1.discovery.steamserver.net	normal
Unknown	Unspecifie	0	0		normal
TLS	Web	1	1		normal
Unknown	Unspecifie	0	0		syn_flood
Unknown	Unspecifie	0	0		syn_flood
Unknown	Unspecifie	0	0		syn_flood
Unknown	Unspecifie	0	0		syn_flood
Unknown	Unspecifie	0	0		syn_flood
Unknown	Unspecifie	0	0		syn_flood
Unknown	Unspecifie	0	0		syn_flood
Unknown	Unspecifie	0	0		syn_flood
Unknown	Unspecifie	0	0		syn_flood
Unknown	Unspecifie	0	0		syn_flood

(2). 訓練預處理

在進行模型訓練之前，我們首先執行了資料預處理流程，以確保輸入資料的品質能夠支撐後續模型的準確訓練與有效學習。第一步是對原始的 CSV 資料進行清洗與轉換。我們先將所有特徵欄位統一轉換為數值型態，這是因為大多數機器學習與深度學習模型無法直接處理文字或類別資料，因此必須將其轉為可計算的數值型格式。對於非數值資料或可能含有錯誤與缺失的欄位，我們透過 `pd.to_numeric()` 自動強制轉換並補上缺失值（如以 0 填補），避免後續模型訓練過程中出現錯誤或資料不一致的狀況。

接下來，我們移除了與模型學習無關的欄位，例如原始封包中的時間戳記、來源與目的 IP、MAC 位址等識別性欄位，這些資訊雖然在網路傳輸層級上可能有意義，但在模型學習的角度中屬於低關聯性特徵或高維雜訊，可能導致過擬合或降低模型泛化能力。最後，我們將資料中的標籤欄位（Label）轉換為二元格式，定義 0 代表正常流量，1 則代表異常流量。這個步驟不僅讓模型能進行二分類學習，也讓後續的性能評估（如 Precision、Recall、F1-score）能更清楚反映出異常偵測任務的效果。

完成初步的資料清理後，我們使用了 `SelectKBest` 函數，搭配 F-score 統計量作為特徵選擇依據。F-score 是一種衡量特徵與標籤之間差異性的重要統計指標，數值越高代表該特徵在不同類別（正常與異常）之間的區分能力越強。我們從所有特徵中選出對分類最具貢獻的前 50 個特徵，以濃縮資訊、降低維度、排除噪音，讓模型專注學習在真正重要的特徵上。接著，我們對這些挑選出的特徵進行標準化處理，使每個特徵的數值分佈轉換為平均值為 0、標準差為 1 的標準常態分佈。這個步驟能避免特徵間的數值尺度差異影響模型學習效

果，並有助於加快模型收斂速度、穩定訓練過程。

資料處理完成後，我們將其轉換為時間序列格式。具體來說，我們將每 10 筆封包資料作為一個輸入序列，目標是預測第 11 筆封包的流量類別（正常或異常）。透過這種方式，模型能夠從連續封包之間學習出時間上的關聯性與變化趨勢，而非僅從單筆封包中進行靜態判斷。這在異常流量偵測的應用中尤為關鍵，因為許多異常行為往往是時間累積下的結果，例如突發連線、封包重複、大量小流量等攻擊模式，通常需要觀察一段時間的流量變化才能辨識。

為了讓模型的訓練與測試更貼近實際情境，我們進一步依照時間順序將資料切分為 60% 的訓練集與 40% 的測試集。與隨機分割不同，這種方式保留了資料的時序性，使模型在學習階段不會提早接觸未來的資料，也更符合「先觀察、再預測」的實務應用場景。這種設計有助於提升模型在實際部署後的預測可信度與穩定度，特別是在即時偵測與線上更新系統中，模型的時序一致性是非常重要的環節。

總結來說，我們在資料處理與轉換階段不僅遵循機器學習的最佳實踐，也根據異常流量特性的需求，設計了兼顧特徵選擇、標準化、時間序列建構與時序切割的完整流程，為後續模型的訓練與即時應用奠定了良好且穩固的基礎。

(3). 訓練的結果分析

從實驗結果來看，我們所設計的模型在分類性能方面展現了非常優異的表現，無論是整體準確率或是針對各類別的分類指標，都達到了高水準的成果。根據 Figure 1 所示，我們的模型在測試資料上的整體準確率高達 0.97，代表模型對於輸入資料的辨識與預測能力相當可靠，能有效區分正常與異常的網路流量。更值得一提的是，針對「異常類別」這種資料不平衡問題常見的挑戰，我們的模型也能保持良好的辨識性能，從每個類別的 F1-score 來看，平均值皆超過 0.95，顯示模型在 Precision 與 Recall 之間取得了高度平衡，不僅能準確預測異常事件，也不容易漏判，展現出高度的實用性與穩定性。

此外，我們在 Figure 2 中繪製了模型的 PR Curve，用來更細緻地觀察模型在不同判斷門檻下的表現。從圖中可以清楚看出，整體曲線幾乎貼近右上角，這表示模型在保持高召回率的同時，依然能維持極高的精確率。換句話說，模型不僅偵測到大部分的異常事件，還能大幅減少誤報，這在實際應用上尤其重

要。因為在現實場景中，若模型誤判率過高，會導致使用者疲於應對虛假警報，降低整體系統的信任度與實用性。

此外，高品質的 **PR Curve** 也意味著模型具有良好的容錯能力與適應性，當環境中的異常類型、流量模式或封包結構發生變化時，模型仍能穩定維持一定水準的偵測效能。這樣的特性使我們的模型非常適合應用在動態變化頻繁的無線設備環境中，例如智慧家庭、IoT 網路或企業內部的 Wi-Fi 系統中，協助即時監控流量並提前預警可能的網路攻擊或異常行為。

總體而言，從各項指標來看，我們所設計的 **CNN-LSTM** 模型不僅具備高準確度與分類效能，還兼顧穩定性與實務應用的可行性，是一個具有高度潛力的異常流量偵測解決方案。

```
=== Classification Report (CNN-LSTM) ===
```

	precision	recall	f1-score	support
ack_flood	0.91	0.98	0.94	2479
fin_flood	0.92	0.98	0.95	1843
normal	0.99	0.96	0.98	22416
rst_flood	0.89	0.99	0.94	1318
syn_flood	0.87	0.98	0.92	1577
accuracy			0.97	29633
macro avg	0.91	0.98	0.95	29633
weighted avg	0.97	0.97	0.97	29633

Figure 1 Classification Report

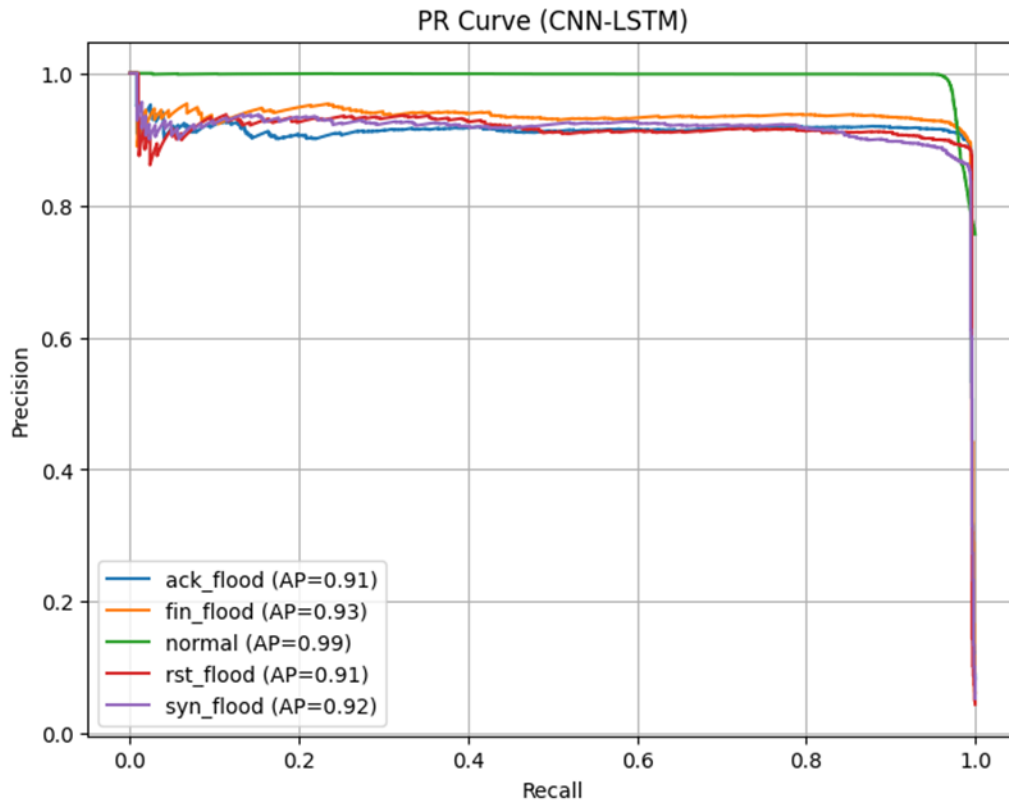


Figure 2 PR Curve

三. 利用 *model* 偵測及其他延伸應用

(1). 利用 *model* 偵測前的處理

要利用 *model* 偵測前，要將事先訓練好的深度學習模型從檔案載入到記憶體中，讓程式可以即時對網路流量進行特徵分析與攻擊偵測

```
k_best = joblib.load('D:\\test_program\\final_project\\final_6\\kbest.save')
model=load_model('D:\\test_program\\final_project\\final_6\\cnn_lstm_model.h5')
scaler = joblib.load('D:\\test_program\\final_project\\final_6\\scaler.save')
le = joblib.load('D:\\test_program\\final_project\\final_6\\label_encoder.save')
```

然後需要依照 *model* 的特徵去讀取對應欄位的 *flow* 資訊，

```

X = pd.DataFrame([feature_dict])

#根據之前的k_best選擇前k重要的特徵，並從x中選出來
X_selected = k_best.transform(X)
#根據scaler來標準化X的特徵
X_scaled = scaler.transform(X_selected)

```

然後因為 model 的輸入格式為(batch size, time_steps , features)，因此需要多筆 flow 的資訊結合成一個序列，還要擴展其維度，才能輸入進 model 內，

```

#因為model的輸入為(幾組序列(batch size), time_steps , features)，所以需要擴展維度
X_input = np.expand_dims(input_seq, axis=0) #axis=0代表在最前面增加一個維度
y_probs = model.predict(X_input)[0] # shape: (3,)

```

model 輸出則會是個類別的機率，類別的順序分別為 ACK flood、FIN flood、normal、RST flood、SYN flood

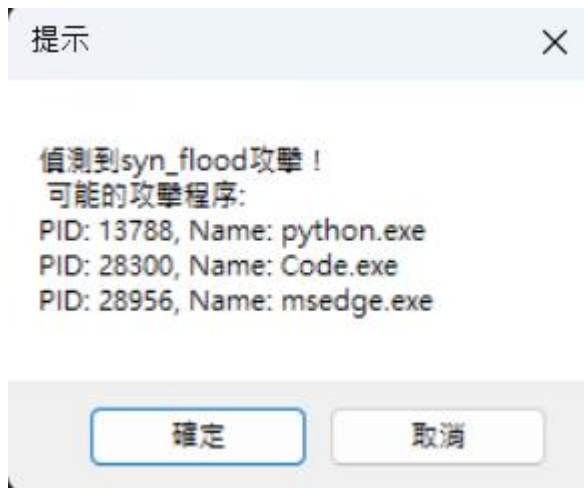
```

evaluate the model.
1/1 ----- 1s 581ms/step
Predicted probability of attack: [4.9151992e-03 2.3368031e-03
9.9056238e-01 5.8187934e-04 1.6037913e-03]
1/1 ----- 0s 24ms/step
Predicted probability of attack: [4.8357113e-03 2.9156262e-03
9.8939741e-01 6.9147156e-04 2.1597142e-03]
1/1 ----- 0s 25ms/step
Predicted probability of attack: [2.2773242e-03 2.7122558e-03
9.9083686e-01 8.2717388e-04 3.3464315e-03]
1/1 ----- 0s 27ms/step
Predicted probability of attack: [1.8606848e-03 1.1470213e-03
9.9480951e-01 4.5484857e-04 1.7278625e-03]
1/1 ----- 0s 31ms/step
Predicted probability of attack: [5.0378358e-04 4.4331895e-04
9.9841380e-01 2.2911638e-04 4.0989608e-04]

```

(2). 偵測到異常後的應用

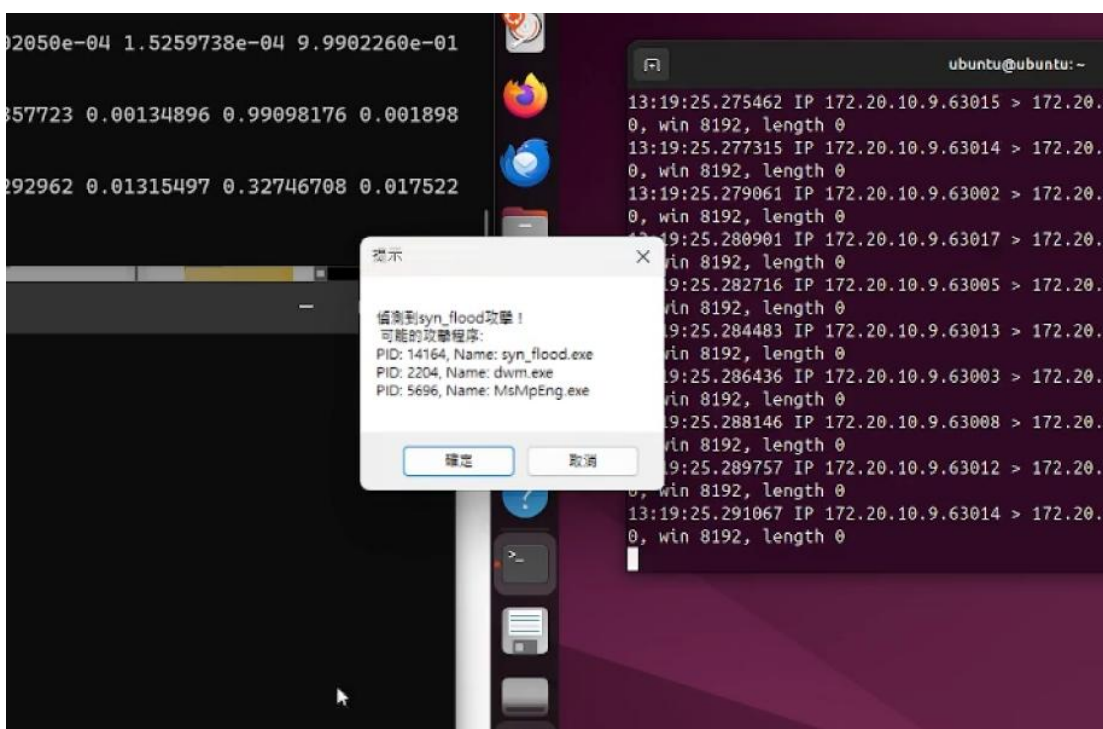
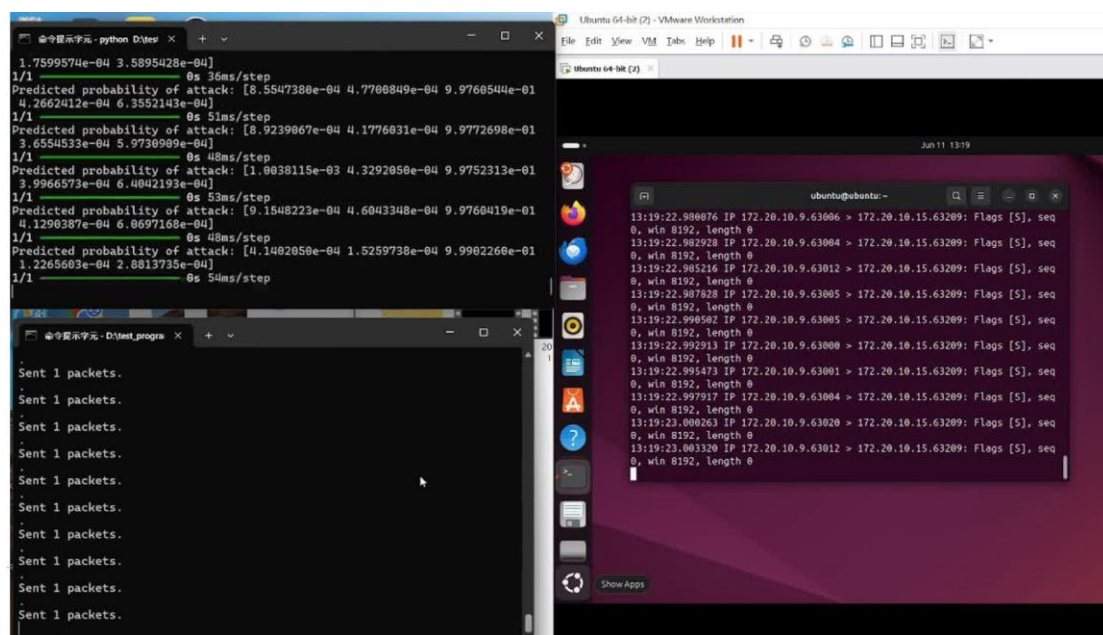
若 model 偵測到異常後，我們做的應用是讓程式跳出一個彈窗，上面會顯示攻擊的類別，還有可能誰製造了這筆異常流量，下圖是偵測到 python 執行 syn_flood 腳本的結果



這邊找出誰製造這筆異常流量的部分，因為無法直接從封包推出是從哪一個 **process** 製造的，而且因為我們製造異常流量是用 **npcap** 這個驅動程式去發送封包，他是繞過 **TCP stack** 來發送封包，所以無法直接用連線的資訊去得到製造異常流量的 **process**，雖然應該可以直接看誰使用了 **npcap**，但為了符合較真實的情況惡意程式不一定使用的是這個驅動程式，所以我們直接使用 **CPU** 增加率作為判別標準，但不一定完全準確，這部分則是未來需要去優化的。

四. Experiment

在 DEMO 時，我們將攻擊的腳本 **syn_flood.py**、**fin_flood.py**、**rst_flood.py**、**ack_flood.py** 都轉成執行檔，方便我們驗證找來源 **process** 的功能。視窗總共有三個，如下圖，左上角是檢測每筆 **flow** 的程式，左下角是發送異常流量的程式，右邊是作為攻擊目標的虛擬機，會顯示傳來封包的資訊，我們虛擬機的 OS 使用的是 **Ubuntu**。



五. Future work

雖然這個專題已經達到偵測四種 TCP flood 的功能，但現實中的攻擊方式遠遠不止四種。而且我們偵測的攻擊模式偏好 burst 類型，若攻擊的頻率較低，就難以偵測出攻擊行為，這邊若要優化的話，可能需要收集更多資料。

除此之外，我們判別異常流量來源的方法是根據 CPU 增加量列出可疑的 process，這個方法也不一定完全準確，也許可以利用機器學習或深度學習的方法去優化這部分。

六. 心得

(1). 謝育葦

在這次的專題製作中，我學到了非常多課堂外不容易接觸到的實作知識與技能。最有收穫的部分，是在實際處理網路連線資料與應用深度學習模型上，從理論到實作都有完整的理解。首先，透過設計與開發即時流量監控與分析系統，我深入理解了網路連線的基本運作原理，包含 TCP 的三次握手以及封包中各種欄位的意義。這不僅加深我對網路協定的理解，也讓我能夠用程式撰寫方式實際擷取與記錄 flow 資料，並進一步分析其是否為潛在的異常流量。再來，這次專題讓我有機會接觸並實際應用深度學習技術。我們使用 CNN 與 LSTM 結合的模型來偵測異常流量，這讓我學會了如何將原始的 flow 特徵轉換成適合模型訓練的格式，也更理解了時間序列與空間特徵的差別與互補性。在訓練與測試模型的過程中，我也學到了模型的結構設計、參數調整、以及模型評估的方式，像是 precision、recall 與 F1-score 等，讓我對機器學習與深度學習有了更扎實的基礎。此外，實作中遇到的問題也讓我學會了解決問題的能力，例如 flow 資料即時擷取失敗、模型無法正確分類、特徵選擇不佳等，每次 debug 與嘗試不同方法的過程中，我都學到了如何查資料、測試假設、並不斷優化系統。最後，也感謝我的組員在專題過程中的互相幫助與合作。

(2). 劉姿妤

透過這次的專題製作，我不僅深入學習了 LSTM 的底層結構與其在時間序列資料中的應用方式，也實際參考並實作了一篇研究論文中提出的 CNN-LSTM 結合架構，這讓我對於如何設計一個能有效處理時間與空間特徵的異常流量偵測模型，有了更全面且具體的理解。在模型架構的建構過程中，我學會了如何根據資料特性選擇合適的特徵處理方法、如何進行序列資料的格式轉換，以及如何評估模型在實務應用上的可行性與表現指標，這些都是我在過去課堂中較少接觸到的實戰經驗。

此外，異常流量偵測、封包分析甚至殭屍網路等主題，原本對我而言相對陌生，但透過這次專題，我不但在理論上建立起對這些網路安全議題的基礎認識，也實際動手從資料擷取、預處理到模型訓練與測試，完整體驗了資料科學技術在資安領域中的應用流程與挑戰。這讓我深刻感受到，資料分析與機器學習不僅能用於傳統領域如金融、商業，也能在網路安全這種高即時性、高風險的場景中發揮關鍵作用。最後，我也要特別感謝我的組員，在每一次的討論中都提供了寶貴的意見與實作協助，讓我們可以順利地完成一個具有完整流程、真實資料與應用場景的異常流量偵測系統。

Reference

[1] Halbouni, A., Gunawan, T. S., Habaebi, M. H., Halbouni, M., Kartiwi, M., & Ahmad, R. (2022). CNN-LSTM: hybrid deep neural network for network intrusion detection system. *IEEE Access*, 10, 99837-99849.